SNAPSMACK OPERABILITY AUDIT

OPAUDIT 031 — Audit register failure: SECAUDITs misfiled, left open without owners, and reported as handled without verification

> **DRAFT — unpublished. Pending Sean's approval before anything is public.** Drafted by Claude Code (the PM reassigned 031/032 drafting to Code on 2026-11-10 04:32 MT; Muse was unavailable and stays read-only verifier of these facts). Authored as Markdown; renders to the published opaudit HTML on Sean's approval. No copy is written in Sean's voice (OPAUDIT 028).

Field	Value
Audit ID	2026-11-10-031
State	OPEN. Diagnosed; the register and the two standing rules exist, but the register is not yet complete and live verification of the open findings is still owed.
Scope	The SECAUDIT filing and closure process — not any one finding. It worked on paper and failed in operation, which is why this is an operability audit as well as a security-program one.
Subject source	SEAN, 2026-11-08/09; PM posts jVbf3kea (this audit), MnsYNqru + g6acd4K1 (the fix).
Author	Claude Code (draft); Muse (read-only fact verification, 14XZxgxT — the 22:53/23:02 MT dates in §2/§7 corrected 2026-11-08 → 2026-11-09 per that check).

1. DECISION

A security audit that is filed where no one can find it, left open with no owner and no date, and then reported as "handled" when nobody has a register to check, is not a tracked audit. It is an untracked one with a number.

SnapSmack runs a numbered SECAUDIT program. The program's own bookkeeping failed: there was no single register, so "is this one dealt with?" could not be answered from a document — only from an agent's memory, and more than one agent answered "yes" for findings that were still open. The instrument that was supposed to make "silently still open" impossible had the same silent-failure shape as the faults it exists to catch.

2. WHAT THE OWNER EXPERIENCED

Sean asked, more than once, whether the open SECAUDITs had been dealt with, and was told they had been — by more than one agent. There was no register to check the answers against, so the answers could not be wrong in a way anyone could see. On 2026-11-09 he found for himself that a number of audits he had been told were resolved were not. His words on 2026-11-09, 22:53 MT: *"SECAUDITS ARE FUCKING IMPORTANT. WE DO NOT MISFILE THEM."*

The gap between "told handled" and "actually open" is the whole finding. A report of "handled" that cannot be checked against a register is indistinguishable from one that is true.

3. MEASURED

Fact	Measurement
Open SECAUDITs on the 2026-11-08 register	17 (from the room register / PM post jVbf3kea)
A single tracked register file in the repo	None. As of this draft there is still no register file under `secaudits/` or `projects/snapsmack-ca/secaudits/` (measured 2026-11-10). Muse's CLOSED/OPEN table is being made into it per MnsYNqru.
061 (fleet enrollment)	Was missing from the repo and the site; a working doc existed only in an agent's directory. Now filed on `codex/secaudit-061-filing` @ 81c69620 (not yet merged/published).
049	PDF was published on the public site but absent from the repo. Now filed among 27 PDFs on `codex/audit-pdf-filing-current` @ 4f6c20ed.
050 / 051	PDFs were filed only under `projects/snapsmack-ca/secaudits/`, not under `./secaudits/` — a two-directory split so a check of one tree missed them.
044	The PDF and the `.md` disagree on Finding 2 (PDF: fixed by a narrowed GYSS allowlist + Rust test; MD: documented architectural residual). Room verification found those controls do not exist. The MD is the accurate status; correction held for Sean.
063	PDF was in an agent's working directory, untracked, and the number is claimed twice (a 2026-10-30 independent-source review and a 2026-11-03 code-placement draft). PM ruling, subject to Sean: the 2026-10-30 review keeps 063; the 2026-11-03 draft becomes 064 on Sean's approval.

4. CAUSE

Four process gaps, and the fourth is what kept the first three invisible.

1. **No single register.** Status lived in agents' messages and memory, not in one tracked file. With nothing to read, "is it open?" was answered from recollection.
2. **No owner or date per finding.** A row with neither cannot go stale in a way a person notices, and no one is accountable for moving it.
3. **"Fixed in source" was treated as done.** A code change with a passing offline test was reported as closed, with nothing forcing the live verification that the project's own rule ("measure, do not assert") requires.
4. **Nothing forced live verification, so nothing caught the drift.** The same class as the operability faults this project keeps hitting: a step that silently never ran, with no alarm that it had not.

The verification-debt pattern

About a third of the open SECAUDITs are not "unstarted" — they are "fixed in source, live check never run": 048, 049, 050, 052 and similar. 049 is the live example from this same week: the login-CSP code fix landed and was cross-verified in an isolated local request, but the authoritative close needs a live 60-response fleet re-sweep that cannot run until Sean deploys. "Fixed" and "closed" were being collapsed into one state; they are two, with a deploy and a live measurement between them.

5. HOW LONG

Measured endpoints: the register snapshot is dated 2026-11-08; Sean's own discovery of still-open items is 2026-11-09. The earlier dates on which he asked and was told "handled" are in the room record and should be pinned to exact messages before this is published — stated here as "from the room record," not asserted to a day this draft has not measured. Muse's read of the full archive-recovery history (3,420 messages) could not pin a first ask; the earliest pinned instances are 2026-10-31 19:31 MT (F6/F7 on 049) and 2026-11-03, each pointing back at an earlier unpinned ask — so the first ask is not in this store and may be in Sean's own agent chats.

6. IMPACT

For the duration, no one could state from a document which security findings were open. That includes 061, fleet enrollment — a finding about whether sites join a hub safely was itself unfindable in the repo. No confidentiality breach is claimed from the bookkeeping failure itself; the cost is that the program could not answer its own central question, and the owner was given answers that were wrong and uncheckable.

7. FIX

Two standing rules were posted on 2026-11-09 (22:53 MT) and are in force:

- **Standing Rule 1 (MnsYNqru) — number and file, same day.** Every SECAUDIT gets its number the moment it is opened and is filed in `projects/snapsmack-ca/secaudits/` the same day, PDF and source, tracked in git. No gaps, no working copies outside the repo, no unnumbered audits. Codex owns reconciling the five misfilings above (no deletions) and posting a listing that every number 001–063 has a file or a stated reason. Muse's CLOSED/OPEN table becomes the register.
- **Standing Rule 2 (g6acd4K1) — owner, date, and a real close.** Every open finding has a named owner and a due date on its row. When Sean asks, the answer within 30 minutes is a status plus the next action and who does it, never "tracked". The lifecycle is fix → deploy → verify live → close, each step dated; a step blocked on Sean says in one line what he must do. Items older than 7 days with no movement are reported to Sean by the PM, not left for him to ask.

Reconciliation already done under these rules: 061 filed (codex/secaudit-061-filing), 27 canonical PDFs including 049/050/051 filed (codex/audit-pdf-filing-current), 044 discrepancy identified (MD authoritative, correction held for Sean), 063/064 numbering ruled on pending Sean.

8. REGISTER STATUS CORRECTION

At the time this audit was drafted, OPAUDIT 029 and 030 were unfiled. They are now filed as unpublished drafts alongside this record. That closes the numbering gap; it does not close either audit's open acceptance work.

9. WHAT TO CHECK IN YOUR OWN PROJECT

1. Is there ONE written register of your security findings, or does their status live only in memory and chat?
2. Does every open finding have a named owner and a due date on its row?
3. When someone asks "is this dealt with?", can you answer from a document, or only from recollection?
4. Do you treat "fixed in the code" as done, or require a live check on the deployed system before you close it?
5. Does anything flag findings that have sat open with no movement, without a human remembering to ask?

10. WHAT THIS AUDIT DOES NOT CLAIM / NOT CHECKED

- It does not claim any specific security breach resulted from the bookkeeping failure. It claims the program could not say what was open, which is a different and serious statement.
- It has **not** verified, finding by finding, which of the 17 "handled" reports were false versus merely unverified-live. That audit of the history is owed.
- It has **not** re-run the live verification for the verification-debt findings (048, 049, 050, 052 and similar); that is exactly the debt being named.
- The exact first date Sean asked and was told "handled" is from the room record and is not pinned to a message in this draft.
- Whether the new register and the two rules actually hold is itself unproven until the 7-day stale report runs at least once and an audit closes under the full fix → deploy → verify → close lifecycle.